Декодирование по максимуму правдоподобия и алгоритм Витерби

05.03.2026

1 Декодирование по максимуму правдоподобия

1.1 Основные обозначения

ДСК (дискретный симметричный канал):

$$\{p(y \mid x),\, x \in X,\, y \in Y\}$$

Кодовая книга:

$$C = \{c_i,\, i = 1, \ldots, M\} \subset X^n$$

Принятый вектор:

$$\bar{y} = (y_1, y_2, \ldots, y_n)$$

1.2 Функции правдоподобия

Функция правдоподобия:

$$p(\bar{y} \mid \bar{c}_i), \quad i = 1, \ldots, M$$

Для независимых каналов:

$$p(\bar{y} \mid \bar{x}) = \prod_{i=1}^{n} p(y_i \mid x_i)$$

Логарифмическая функция правдоподобия:

$$\Lambda_m(\bar{y}) = \ln p(\bar{y} \mid \bar{c}_m) = \sum_{i=1}^{n} \underbrace{\ln p(y_i \mid c_{mi})}_{\lambda_m(y_i)} = \sum_{i=1}^{n} \lambda_m(y_i)$$

1.3 Отношение правдоподобия

$$L(y) = \ln \frac{p(y \mid 1)}{p(y \mid 0)} \quad \begin{cases} > 0, & y = 1 \\ < 0, & y = 0 \end{cases}$$

Вектор отношений правдоподобия:

$$L(\bar{y}) = \big(L(y_1), L(y_2), \ldots, L(y_n)\big)$$

Вспомогательный вектор ξ_m:

$$\xi_m = (\xi_{m1}, \xi_{m2}, \ldots, \xi_{mn}), \quad \text{где} \quad \xi_{mi} = 2c_{mi} - 1 = \begin{cases} -1, & c_{mi} = 0 \\ +1, & c_{mi} = 1 \end{cases}$$

1.4 Теорема

Теорема 1. Декодирование по максимуму правдоподобия сводится к поиску кодового слова, для которого **минимально евклидово расстояние**.

Скалярное произведение:

$$\Lambda_m(\bar{y}) = (\xi_m, L(\bar{y}))$$

Для двух различных кодовых слов $\bar{c}_m$ и $\bar{c}_{m'}$ $(m \neq m')$:

$$\Lambda_m(\bar{y}) \geq \Lambda_{m'}(\bar{y}) \iff L_m(\bar{y}) \geq L_{m'}(\bar{y})$$

Разность логарифмических правдоподобий:

$$\begin{aligned} L_m(\bar{y}) - L_{m'}(\bar{y}) &= \sum_{j \in I_{10}} \big[(+1) - (-1)\big] \cdot L(y_j) + \sum_{j \in I_{01}} \big[(-1) - (+1)\big] \cdot L(y_j) \\ &= 2 \sum_{j \in I_{10}} L(y_j) - 2 \sum_{j \in I_{01}} L(y_j) \end{aligned}$$

где I_{10} — позиции, где $c_{mj} = 1, c_{m'j} = 0$, а I_{01} — наоборот.
Продолжая преобразования:

$$= 2 \sum_{j \in I_{10}} \ln \frac{p(y_j \mid \bar{c}_{mj})}{p(y_j \mid \bar{c}_{m'j})} - 2 \sum_{j \in I_{01}} \ln \frac{p(y_j \mid \bar{c}_{mj})}{p(y_j \mid \bar{c}_{m'j})} = 2 \sum_{j=1}^{n} \ln \frac{p(y_j \mid \bar{c}_{mj})}{p(y_j \mid \bar{c}_{m'j})}$$

1.5 Связь с расстоянием Хэмминга

Для ДСК с вероятностью ошибки p_0:

$$L(y) = \ln \frac{p(y \mid 1)}{p(y \mid 0)} = \begin{cases} \ln \dfrac{1 - p_0}{p_0} = \delta, & y = 1 \\ \ln \dfrac{p_0}{1 - p_0} = -\delta, & y = 0 \end{cases}$$

где $\delta = \ln \frac{1-p_0}{p_0} > 0$ при $p_0 < \frac{1}{2}$.
Тогда:

$$L_m(\bar{y}) = (\xi_m, L(\bar{y})) = \sum_{i=1}^{n} (2c_{mi} - 1) \cdot L(y_i)$$

Обозначим $d(\bar{y}, \bar{c}_m)$ — расстояние Хэмминга между $\bar{y}$ и $\bar{c}_m$.
При $y_i = c_{mi}$ вклад равен δ, при $y_i \neq c_{mi}$ — $-\delta$:

$$\Lambda_m(\bar{y}) = d(\bar{y}, \bar{c}_m) \cdot (-\delta) + \big(n - d(\bar{y}, \bar{c}_m)\big) \cdot \delta = \delta\big(n - 2d(\bar{y}, \bar{c}_m)\big) \to \max$$

При $p_0 < \frac{1}{2}$, $\delta > 0$, поэтому максимизация эквивалентна:

$$d(\bar{y}, \bar{c}_m) \to \min$$

Вывод: Декодирование по минимуму расстояния Хэмминга = декодирование по максимуму правдоподобия в ДСК.

2 Поиск кратчайшего пути в решётке. Алгоритм Витерби

2.1 Свойства решётки

1. Вершина находится на пересечении уровня (яруса, уровня)
2. Нулевой и последний ярусы содержат по 1 узлу
3. Решётка — направленный граф без циклов
4. Рёбра помечены метриками (весами)
5. Длина пути $= \sum$ весов рёбер, входящих в путь

2.2 Пример: код с проверкой на чётность

Порождающая матрица:

$$G = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

$(3, 2)$-код с проверочной матрицей на чётность.
Кодовые слова: $000, 011, 110, 101$.

3 Треллис-диаграммы и алгоритм Витерби

3.1 Принцип построения

Чем меньше узлов/вершин, тем эффективнее декодер.

Сложность решётки определяется её профилем:

$$\nu = (\nu_0, \ldots, \nu_n)$$

Пример:

4 Алгоритм Витерби

Алгоритм Витерби — это метод динамического программирования для нахождения наиболее вероятной последовательности состояний (пути) в решётчатой диаграмме. Он широко применяется при декодировании свёрточных кодов, а также в задачах обработки сигналов и распознавания речи.

4.1 Основные понятия

Рассмотрим решётку с L ярусами (длина кодового слова). На каждом ярусе $t = 0, 1, \ldots, L$ имеется множество состояний S_t. Каждому переходу из состояния s_{t-1} в состояние s_t приписана **метрика ветви** $\gamma_t(s_{t-1}, s_t)$, которая характеризует «стоимость» этого перехода. В задаче декодирования по максимуму правдоподобия метрика ветви обычно равна логарифму правдоподобия принятого символа при условии переданного кодового символа, либо (для ДСК) расстоянию Хэмминга между переданным и принятым битами.

Для каждого состояния s_t вводится **накопленная метрика** $\Gamma_t(s_t)$ — минимальная (или максимальная) сумма метрик ветвей среди всех путей, ведущих из начального состояния s_0 в s_t. В зависимости от постановки задачи мы либо минимизируем суммарное расстояние, либо максимизируем правдоподобие. Будем далее считать, что цель — минимизация суммарной метрики (например, расстояния Хэмминга).

4.2 Рекуррентная процедура

Алгоритм работает в три этапа: инициализация, прямой ход (рекурсия) и обратный ход (traceback).

1. **Инициализация.** Задаём начальную метрику для стартового состояния:

$$\Gamma_0(s_0) = 0, \quad \Gamma_0(s) = +\infty \text{ для остальных состояний.}$$

2. **Прямой ход.** Для каждого яруса $t = 1, \ldots, L$ и для каждого состояния s_t вычисляем:

$$\Gamma_t(s_t) = \min_{s_{t-1} \in \mathcal{P}(s_t)} \big(\Gamma_{t-1}(s_{t-1}) + \gamma_t(s_{t-1}, s_t)\big),$$

   где $\mathcal{P}(s_t)$ — множество предшественников состояния s_t. Одновременно запоминаем указатель на того предшественника, на котором достигается минимум (выживший путь).

3. **Завершение.** На последнем ярусе L выбираем состояние s_L^* с минимальной накопленной метрикой $\Gamma_L(s_L^*)$. Это конечная точка наиболее вероятного пути.

4. **Обратный ход.** Двигаясь от s_L^* назад по сохранённым указателям, восстанавливаем всю последовательность состояний (и, соответственно, кодовых символов).

Если решётка имеет финальное состояние (как обычно в свёрточных кодах с принудительным завершением), то поиск ведётся только среди допустимых конечных состояний.

4.3 Пример работы алгоритма

Рассмотрим (3,2)-код с проверкой на чётность, заданный порождающей матрицей

$$G = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

Кодовые слова: 000, 011, 110, 101. Построим для него решётку, в которой каждый путь соответствует одному кодовому слову. Состояниями будем считать значения первого информационного бита u_1 после первого такта и пары (u_1, u_2) после второго такта. На третьем такте все пути сходятся в конечное состояние (кодовое слово завершено). Для простоты изобразим решётку, где на каждом ярусе указаны возможные состояния и переходы с пометками «выходной символ / метрика ветви».

Пусть по каналу связи был передан один из кодовых векторов, а принят вектор $\bar{y} = (1, 0, 1)$. Для ДСК с вероятностью ошибки $p_0 < 1/2$ декодирование по максимуму правдоподобия эквивалентно поиску кодового слова с наименьшим расстоянием Хэмминга до $\bar{y}$. Вычислим расстояния:

- $d(000, 101) = 2$ (позиции 1 и 3) - $d(011, 101) = 1$ (позиция 2) - $d(110, 101) = 2$ (позиции 1 и 2?) - $d(101, 101) = 0$

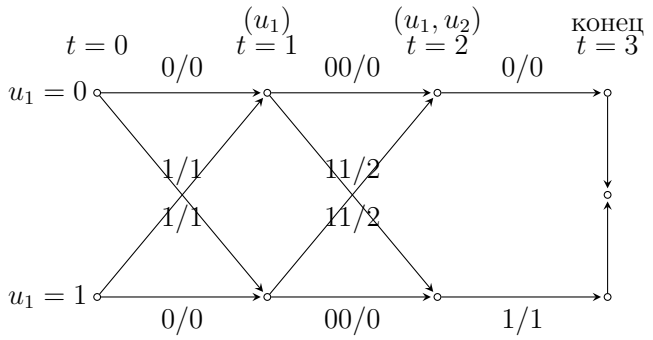

Рис. 1: Решётка для (3,2)-кода. На рёбрах указаны выходные символы (последовательность из трёх битов разбита по тактам) и расстояние Хэмминга до соответствующего символа принятого вектора (пока не задано).

Очевидно, что наилучшее слово — 101. Теперь покажем, как алгоритм Витерби найдёт его, используя решётку и метрики ветвей, равные расстоянию Хэмминга между переданным символом (на ребре) и соответствующим битом $\bar{y}$.

Разобьём $\bar{y}$ по тактам: на первом такте передаётся первый бит кодового слова, на втором — второй, на третьем — третий. Таким образом: - Такт 1: принят $y_1 = 1$. - Такт 2: принят $y_2 = 0$. - Такт 3: принят $y_3 = 1$.

Теперь для каждого ребра вычислим его вес (метрику ветви) как расстояние Хэмминга между выходным символом (указан на рисунке) и принятым битом на этом такте.

Такт 1 (переход из $t = 0$ в $t = 1$): - Из состояния $u_1 = 0$: * Ребро с выходом 0: вес $= |0 - 1| = 1$. * Ребро с выходом 1: вес $= |1 - 1| = 0$. - Из состояния $u_1 = 1$: * Ребро с выходом 1: вес $= |1 - 1| = 0$. * Ребро с выходом 0: вес $= |0 - 1| = 1$.

Такт 2 (переход из $t = 1$ в $t = 2$): здесь на ребрах указаны два бита (второй и третий символы кодового слова? В нашей решётке мы для простоты разбили трёхбитовое слово на три такта по одному биту, но на самом деле код генерирует все три бита одновременно. Чтобы решётка была ацикличной и соответствовала временной последовательности, нужно каждому информационному биту сопоставить свой кодовый символ. В нашем примере кодирование происходит так: $c_1 = u_1$, $c_2 = u_1 + u_2$, $c_3 = u_2$. Тогда на первом такте мы получаем c_1, на втором — c_2, на третьем — c_3. Это позволяет построить решётку с тремя ярусами кодовых символов. Уточним рисунок: на первом ярусе переходы помечены значением c_1, на втором — c_2, на третьем — c_3. Состояния после первого яруса отражают значение u_1, после второго — пару (u_1, u_2).

Перерисуем решётку с правильными пометками:

Для принятого вектора $(1, 0, 1)$ метрики ветвей (расстояние Хэмминга) на каждом такте: - Такт 1 ($y_1 = 1$): * Рёбра с символом 0: вес $= 1$ * Рёбра с символом 1: вес $= 0$ - Такт 2 ($y_2 = 0$): * Рёбра с символом 0: вес $= 0$ * Рёбра с символом 1: вес $= 1$ - Такт 3 ($y_3 = 1$): * Рёбра с символом 0: вес $= 1$ * Рёбра с символом 1: вес $= 0$

Теперь выполним прямой ход алгоритма, заполняя таблицу накопленных метрик $\Gamma_t(s)$.

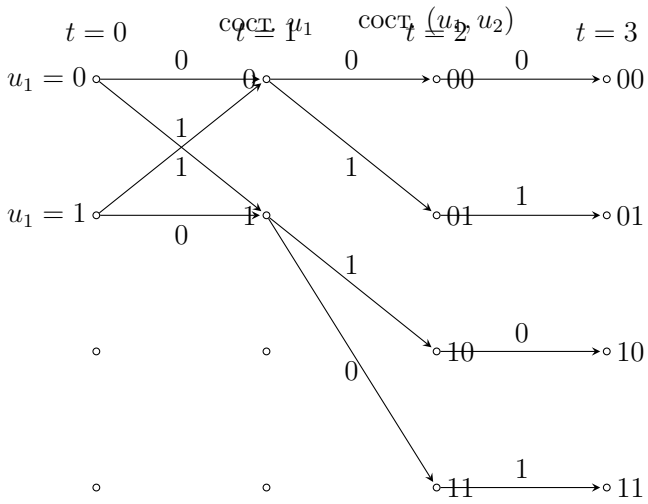

Рис. 2: Корректная решётка для (3,2)-кода с потактовой выдачей кодовых символов c_1, c_2, c_3.

t	Состояние	Предшественники	Накопленная метрика Γ	Выживший предшественник
0	(начало)	—	0	—
1	$u_1 = 0$	из нач. (0/1)	$0 + 1 = 1$	нач. (символ 0)
	$u_1 = 1$	из нач. (1/0)	$0 + 0 = 0$	нач. (символ 1)
2	$(0, 0)$	из $u_1 = 0$ (символ 0/0)	$1 + 0 = 1$	$u_1 = 0$
	$(0, 1)$	из $u_1 = 0$ (символ 1/1)	$1 + 1 = 2$	$u_1 = 0$
	$(1, 0)$	из $u_1 = 1$ (символ 1/1)	$0 + 1 = 1$	$u_1 = 1$
	$(1, 1)$	из $u_1 = 1$ (символ 0/0)	$0 + 0 = 0$	$u_1 = 1$
3	$(0, 0)$	из $(0, 0)$ (символ 0/1)	$1 + 1 = 2$	$(0, 0)$
	$(0, 1)$	из $(0, 1)$ (символ 1/0)	$2 + 0 = 2$	$(0, 1)$
	$(1, 0)$	из $(1, 0)$ (символ 0/1)	$1 + 1 = 2$	$(1, 0)$
	$(1, 1)$	из $(1, 1)$ (символ 1/0)	$0 + 0 = 0$	$(1, 1)$

На последнем ярусе минимальная накопленная метрика равна 0 у состояния $(1, 1)$. Это состояние соответствует информационной паре $(u_1, u_2) = (1, 1)$, которая кодируется в слово $c = (u_1,\ u_1 + u_2,\ u_2) = (1, 0, 1)$. Таким образом, алгоритм Витерби определил кодовое слово 101, что совпадает с прямым перебором.

4.4 Сложность алгоритма

Количество операций на каждом шаге пропорционально числу рёбер в решётке. Если решётка имеет S состояний и из каждого состояния выходит 2^k рёбер (для свёрточного кода со скоростью k/n), то общее число обрабатываемых переходов равно $L \cdot S \cdot 2^k$. Это значительно меньше полного перебора всех 2^{kL} кодовых слов. Именно поэтому алгоритм Витерби широко применяется на практике.